

TA Post Education Learning plan

Web Automation Java	2
Test cases creation	2
Automation on pure selenium	2
Automation on carina	4
Mobile Automation Android (WEB view)	5
Mobile Automation iOS (WEB view)	7
Mobile Automation Native (iOS + Android)	9
API Automation w/o Carina	11
BDD Testing (Cucumber)	12
Advanced Automation Java NEW	13
Infrastructure	14
Docker	14
Selenoid/GGR	17
Jenkins	18
Zebrunner	21
Mobile Cloud	23
Github Actions	24
Useful tools	25
Testrail	25
Jira	25
Proxyman tool	27
Browsermob Proxy (analytics test automation)	28
Advanced Java	30
Lambda functions NEW	30
Threads	30
Regular expressions NEW	31
Manual testing basics	35
Web Automation Cypress/Webdriver IO	36
Performance testing with Gatling	37
Python/PyTest program	38

Web Automation Java

Test cases creation

Task

Create google spreadsheet with test cases for some web application.

Goal: understand how to organize and build test cases; practice with independent test-cases creation.

Details:

Cases number: 5-7

Cases characteristic: high level testing of some major website functionalities.

I recommend using some e-shop website which has grid of products, filtering, basket, account functionalities. Here you will find [websites](#).

Cases should be listed in spreadsheet in format:

Case ID || Title || Steps || Expected results

Materials:

-

Automation on pure selenium

Task 1

Create simple java test project with selenium and TestNG as dependencies.

Goal: learn PageObject and PageFactory pattern, improve knowledge of maven, practice selenium and github usage

Details:

1. Learn about maven archetype, use basic archetype to generate new project

(<https://maven.apache.org/archetypes/maven-archetype-quickstart/>). import it to IDE.

2. Add selenium, testng, slf4j, log4j as dependencies.

3. Automate scenario (example):

- open tested website
- search for some product
- click Search button
- validate that page contains not-empty list of results and print all titles in console

4. Use PageObject and PageFactory patterns for that

5. Use css selectors for your web elements (https://www.w3schools.com/cssref/css_selectors.asp)

6. Start standalone selenium server with chromedriver and debug implemented test on it
7. Create @solvd github account (if not yet created), create new repo, push your project there (use .gitignore not to push test output files)

Materials:

https://www.selenium.dev/documentation/guidelines/page_object_models/
<https://www.browserstack.com/guide/page-object-model-in-selenium>

Task 2

Add abstractions to implemented test scenario (from Task 1).

Goal: better understand and implement parallelization; learn logging usage; practicing with selenium explicit waits

Details:

1. Create AbstractPage; AbstractTest classes with some common logic.
2. Create wrapper for each used selenium WebElement method (like click, sendKeys) and put it into AbstractPage class. That wrapper should additionally have logging (slf4j + log4j) + dynamical waits of web elements (based on WebDriverWait class usage)
3. Implement ability to run tests in parallel mode without conflict of multiple WebDriver instances.

Materials:

<https://www.selenium.dev/documentation/webdriver/waits/>

Task 3

Cover all prepared test cases from Test cases creation task.

Goal: practice with building of automation based on test cases. Understand how to organize test cases to make process of implementation easier.

Details:

1. Extend test project built in Task 1; use abstractions build in Task 2; set up running of tests in several threads

Materials:

<https://www.selenium.dev/documentation/webdriver/waits/>

Task 4

Set up selenium grid and use it; add screenshots

Goal: learn selenium grid structure; set up grid independently

Details:

1. Set up selenium grid and use it execute tests in parallel mode using that grid. Try different browsers at least chrome and FF
2. Add taking of screenshot for every test failure

Materials:

https://www.selenium.dev/documentation/grid/when_to_use_grid/

Automation on carina

Task 5

Migrate earlier implemented tests onto carina usage

Goal: Understand how custom java selenium frameworks could be easily migrated to carina

Details:

Materials:

<https://zebrunner.github.io/carina/>

Mobile Automation Android (WEB view)

Task 1

Adapt WEB tests migrated to Carina for executing in mobile browser (Android).

Goal: Practice with automation tests for Android platform; learn how to set up and use emulators for automation writing and running.

Details:

1. Learn how to use CustomTypePageFactory
(<http://zebrunner.github.io/carina/automation/mobile/> “How to use CustomTypePageFactory”) and make next changes in existent tests: move basic page classes to separate classes and add implementation for 2 platforms: Android and Desktop using this carina approach.
At the beginning all the logic should be placed in Base page classes. Later once you debug tests in Android browser, methods that require changes for mobile browser should be moved to Android classes with specific implementation.
2. Configure Android studio and set up some Android emulator
3. Install and configure Appium
4. Update config.properties of your test project and set up running tests on your local Appium which will execute tests on Android emulator.
Capabilities sample can be found here
<http://zebrunner.github.io/carina/automation/mobile/> and here
<https://appium.io/docs/en/writing-running-appium/web/mobile-web/#android-mobile-web-automation>
5. Mobile view of website getting locators can be found in chrome in mobile emulation view <https://developer.chrome.com/docs/devtools/device-mode/> or Appium inspector can be used <https://github.com/appium/appium-inspector>

Materials:

<http://zebrunner.github.io/carina/automation/mobile/>
<https://appium.github.io/appium.io/docs/en/writing-running-appium/running-tests/>
<https://appium.github.io/appium.io/docs/en/about-appium/getting-started/>
<https://appium.io/docs/en/writing-running-appium/web/mobile-web/>
<https://github.com/appium/appium-inspector>

Task 2

Add switching of contexts for Android view test

Goal: Practice with different contexts in mobile automation

Details:

1. Learn about different context in mobile automation:
<https://appium.readthedocs.io/en/latest/en/writing-running-appium/web/hybrid/> and how they can be switched with carina
<http://zebrunner.github.io/carina/automation/mobile/#how-to-change-context-of-application==>
2. Add some extra-step in your automation that requires switching between mobile contexts. For example start downloading some file at website (which will cause native mobile popup) or try to open new browser tab using native context.

Materials:

<https://appium.readthedocs.io/en/latest/en/writing-running-appium/web/hybrid/>

<http://zebrunner.github.io/carina/automation/mobile/#how-to-change-context-of-application>

Mobile Automation iOS (WEB view)

Task (requires Mac OS environment)

Adapt WEB tests migrated to Carina for executing in mobile browser (iOS).

Goal: Practice with automation tests for iOS platform; learn how to set up and use simulator for automation writing and running.

Details:

1. Learn how to use CustomTypePageFactory

(<http://zebrunner.github.io/carina/automation/mobile/> “How to use CustomTypePageFactory”) and make next changes in existent tests: move basic page classes to separate classes and add implementation for 2 platforms: iOS and Desktop using this carina approach.

At the beginning all the logic should be placed in Base page classes. Later once you debug tests in Safari browser, methods that require changes for mobile browser should be moved to iOS classes with specific implementation.

Note: If same was already done for Android then just add iOS implementation in addition to Desktop/Android.

2. Configure xcode and set up some iOS simulator
3. Install and configure Appium (if not yet done)
4. Update config.properties of your test project and set up running tests on your local Appium which will execute tests on iOS simulator.

Capabilities sample can be found here

<http://zebrunner.github.io/carina/automation/mobile/> and here <https://appium.io/docs/en/latest/guides/caps/>

5. Learn how to build iOS locators using Predicates:

<https://github.com/facebookarchive/WebDriverAgent/wiki/Predicate-Queries-Construction-Rules> and class chains: <https://github.com/facebookarchive/WebDriverAgent/wiki/Class-Chain-Queries-Construction-Rules>

and use them in your automation

6. Mobile view of website getting locators can be found in chrome in mobile emulation view <https://developer.chrome.com/docs/devtools/device-mode/> or Appium inspector can be used <https://github.com/appium/appium-inspector>.

If native locators can't be found using mentioned tools then you can try to use Macaca App inspector: Macaca App Inspector

7.

Materials:

<http://zebrunner.github.io/carina/automation/mobile/>

<https://appium.io/docs/en/writing-running-appium/running-tests/>

<https://appium.io/docs/en/writing-running-appium/web/mobile-web/#mobile-safari-on-simulator>

<https://github.com/appium/appium-inspector>

<https://github.com/appium/appium-xcuitest-driver/blob/master/docs/real-device-config.md>

<https://developer.chrome.com/docs/devtools/device-mode/>

<https://github.com/facebookarchive/WebDriverAgent/wiki/Predicate-Queries-Construction-Rules>

<https://github.com/facebookarchive/WebDriverAgent/wiki/Class-Chain-Queries-Construction-Rules>

<https://macacajs.github.io/app-inspector/>

Mobile Automation Native (iOS + Android)

Task 1. Set up environment

1. Install the latest versions of Appium and Appium Inspector
2. Install needed drivers
3. Install Xcode
4. Install Android studio and SDK
5. Install adb
6. Setup iOS and Android simulators
7. Create a new repository and project base on Carina framework

Task 2. Create test cases

1. In your Zebrunner internal project (in Zebrunner TCM) create at least **10** test cases for [application](#) under the test
2. Share them with your mentor for review

Task 3. Implement test cases

1. Following documentation and best practices implement your test cases **for both platforms (iOS + Android)**
2. Make sure to use CustomTypePageFactory class for building of your page objects. (See Carina -> Mobile Automation doc for more details)
3. Share a PRs with your mentor for review

Task 4. Practice with find by images strategy

1. Check How to use find by image strategy in the documentation (Carina doc)
2. Install appium images plugin
3. Install opencv4nodejs library
4. Create a test case where you can check any image (in the app you can find a DRAWING feature. Try to draw something by coordinates and then check if it works by comparing images)
5. Implement test case above using images search strategy

Materials

Documentation:

- Mobile automation on Carina <https://zebrunner.github.io/carina/automation/mobile/>
- Launch simulator/emulator https://medium.com/@abrisad_it/how-to-launch-ios-simulator-and-android-emulator-on-mac-cd198295532e
- Appium <https://appium.io/docs/en/2.4/>
- Appium inspector <https://github.com/appium/appium-inspector>
- Class-chain locators: <https://github.com/facebookarchive/WebDriverAgent/wiki/Class-Chain-Queries-Construction-Rules>
- Xpath cheatsheet <https://devhints.io/xpath>

App for testing (iOS + Android):

<https://github.com/saucelabs/sample-app-mobile/releases/tag/2.7.1>

API Automation w/o Carina

Task 1

Backend test automation using simple http client + TestNG library

Goal: Practice with backend test automation without Carina usage. Training with necessary libraries searching and applying of them in project.

Details:

1. Use one of available resources from <https://gorest.co.in> (/users, /posts, etc.) and build suite of minimum **10 tests** for mentioned resource. Test all possible HTTP methods, all tests should be 100% independent (all resources used in the test should be created as part of the same test).
2. Prepare another suite of **5 tests** for GraphQL Endpoint from <https://gorest.co.in>.
3. **OAuth 2 authentication** with Bearer token usage should be applied as part of the tests. Check instructions from <https://gorest.co.in> to get more details.
4. Tests should be documented in Zebrunner TCM.
5. Mentor's review is needed before implementation start.
6. Create new Maven project but without Carina in dependencies list. You can use only TestNG library, any http client, any json processing libraries (for comparison or data retrieval).
7. Tests should successfully validate http status/body/headers of your https calls.
8. Request body data (or data for response's body validation) should be stored as a template, do not copy entire requests bodies just adding small changes.

Materials:

1. Bearer token: <https://medium.com/@arunchaitanya/wtf-is-bearer-token-an-in-depth-explanation-60695b581928>
2. GraphQL <https://graphql.org/>

BDD Testing (Cucumber)

Task

Practice with BDD automation

Goal: learn foundations of BDD approach, receive practical experience automating tests on Cucumber.

Details:

1. Learn cucumber documentation here: <https://cucumber.io/docs/cucumber/> and cucumber gherkin syntax: <https://cucumber.io/docs/gherkin/>
2. Generate new carina-based project for cucumber. Follow https://zebrunner.github.io/carina/getting_started/ for instructions (-
DarchetypeArtifactId=carina-cucumber-archetype)
3. Prepare local MySQL/Postgres database with 2 tables: User and User Orders. Insert some amount of data in these tables based on data from <https://www.saucedemo.com/> website. Multiple users with multiple different order should be stored
4. Build carina based tests based on Cucumber which includes next steps:
 - Log in to <https://www.saucedemo.com/> with user credentials taken from data base
 - Add orders which are stored in DB for mentioned user
 - Proceed to checkout
 - Make sure that ordering was successful

Note: Use MyBatis mappers for interaction with DB

5. Automate another scenario in the same feature for different user and his corresponding orders;
6. Check local cucumber report presence

Materials:

- <https://cucumber.io/docs/cucumber/>
- <https://cucumber.io/docs/gherkin/>
- <https://zebrunner.github.io/carina/cucumber/>

Advanced Automation Java **NEW**

Task 1 - TestNG retry logic applying

Create demo project for TestNG IRetryAnalyzer applying.

Details:

Requirements for the project:

1. Build new Java project based on TestNG without Carina. It should include several primitive tests which should fail on purpose
2. Apply IRetryAnalyzer for those tests and add configurable parameter on project level which defines count of retries.
3. Logic should successfully work in multi-thread mode.
4. Applying of this logic in any Test automation project for end-user should be available with minimal efforts (as it currently works in Carina).
5. Integrate test project with Zebrunner and track couple of runs there as a demo.

Materials:

1. https://testng.org/#_rerunning_failed_tests

Task 2 - Automation of email verification

Create an utility to automate email verification. Your utility should connect to the email server and check if a message has been received.

Notes:

Please **don't use your working email and or any other sensitive credentials for this task**. Create any new dummy email address.

Details:

1. Create any new dummy email address on any service.
2. Implement integration with your email server using [Jakarta Mail API](#). Make sure it can work with different email servers.
3. Go to <https://sendtestemail.com/>, write a test to send an email to your email address and verify that test email has been received.

Materials:

<https://jakarta.ee/specifications/mail/2.0/jakarta-mail-spec-2.0>

Infrastructure

Docker

Task 1

Practice with docker basics. Implement isolated test runner

Details:

1. Install docker locally <https://docs.docker.com/engine/install/>
2. Clone carina demo project <https://github.com/zebrunner/carina-demo>
3. Learn about possible Dockerfile instructions
<https://docs.docker.com/engine/reference/builder/>
4. Create image config for isolated running of carina-demo API suite inside of Docker container.

For that:

- Create and configure Dockerfile inside of carina-demo folder
 - Image should be based on JDK-11 linux public docker image
 - Copy carina-demo project files inside your image (COPY instruction)
 - Add maven installation to your image
 - Configure running of the tests with “mvn .. test” command which will start right after creation of new container (use CMD or ENTRYPOINT instructions for that)
5. Build image using “docker build” command
 6. Run container from your custom image using “docker run” command
 7. Check logs of your started container with “docker logs CONTAINER_NAME”
 8. Once tests run is done log in to your docker container and navigate through execution results: “docker exec -it CONTAINER_NAME” to get into container and then navigate in container as in regular linux machine

Materials:

<https://docs.docker.com/get-started/>

<https://docs.docker.com/engine/reference/builder/>

<https://docs.docker.com/engine/reference/commandline/build/>

<https://docs.docker.com/language/nodejs/run-containers/>

<https://devopswithdocker.com/part-1/section-4/>

Task 2

Practice with docker-compose basics

Details:

1. Install docker-compose locally (if not installed with Docker desktop)
<https://docs.docker.com/compose/install/>
2. Move configuration for building of your custom image into docker-compose.yml file
3. Implement in docker-compose.yml sharing of files between container and your computer (using "volumes"). Map "target" folder of carina-demo project to some local folder on your computer.
So that once test run is completed you need to see "target" folder appeared in your local file system containing carina's html report with result.
4. Set up passing of environment variables via "docker-compose.yml". Use passed environment variables in your "mvn test" command
5. Run your container with docker-compose: "docker-compose up -d"
6. Check target folder presence in your local file system after finishing of test run.
7. Check logs of started container

Materials:

<https://docs.docker.com/compose/>

<https://docs.docker.com/compose/gettingstarted/>

<https://docs.docker.com/compose/environment-variables/>

Selenoid/GGR

Task

Use Selenoid for your tests execution

Details:

1. Learn Selenoid documentation <https://aerokube.com/selenoid/latest/>
2. Set up Selenoid in local Docker using <https://docs.docker.com/compose/> Configure at least FF, Chrome and Edge (on Mac M1 you will be limited by browsers images usage - see Note for details). Create browsers.json file manually and specify all browsers configurations there.
3. Set up Selenoid UI to be able to view execution of your test via VNC
4. Run your automated tests on configured Selenoid. Monitor execution with Selenoid UI.
5. Configure recording of test sessions video to your local environment (NOT in container only). Set up docker volume for that. It will connect some folder on your computer to videos folder in container.
6. Set up GGR on the top of your Selenoid container <https://aerokube.com/ggr/latest/> It's also working in Docker. Use same docker-compose.yml configuration file for it (as for selenoid)
7. Execute your test on GGR. Check that all browsers are successfully working. Execution will happen in Selenoid containers
8. Additionally include your local Selenium grid (NOT Selenoid!) into GGR configuration. Check that you can run tests on Selenium (running without Docker) via GGR url

Materials:

<https://aerokube.com/selenoid/latest/>

<https://aerokube.com/ggr/latest/>

Note:

If you work on Mac M1 then you'll need chrome docker image applicable for this processor architecture.

It can be found by following link: <https://github.com/aerokube/images/pull/524>

Put valid details into your `browsers.json` to make it working (for now chrome v.100 is available).

On Mac M1 Edge browser can be ignored - just set up Chrome and Firefox runs.

Jenkins

Task 1

Practice with Jenkins installation and usage

Details:

1. Using Docker and guide from github deploy Zebrunner Jenkins to your local environment: <https://github.com/zebrunner/jenkins-master>

Warning: There can be issues with Carina version 1.2.6 and higher. The following properties ensure compatibility:

<properties>

<project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>

<carina-core_version>1.1.5</carina-core_version>

<carina-dataprovider.version>1.1.3</carina-dataprovider.version>

<carina-api.version>1.1.3</carina-api.version>

<carina-aws-s3.version>1.1.4</carina-aws-s3.version>

<carina-azure.version>1.1.4</carina-azure.version>

<carina-appcenter.version>1.1.5</carina-appcenter.version>

<java.version>11</java.version>

<zebrunner-agent.version>1.9.7</zebrunner-agent.version>

<suite>helloWorld</suite>

</properties>

2. Create some new Jenkins job that will:
 - Clone carina-demo repository
 - Will execute API sample suite in Jenkins
 - Will publish HTML carina report as artifact for your job
3. Run the job, check its logs and published artifacts.
4. Try to set scheduled execution time and see how it's applied

Note: You can add your personal github credentials into Jenkins using Manage Jenkins - Credentials.

Notes:

You may face the problem when your Docker container with GGR/Selenoid is not accessible from Jenkins container via localhost. Then read following article and make appropriate changes in your configuration: <https://www.cloudsavvyit.com/14114/how-to-connect-to-localhost-within-a-docker-container/>

Materials:

Task 2

Practice with qps-pipeline and Jenkins Job DSL usage

Details:

1. In your test project (implemented in scope of tasks from Web Automation sections) add Jenkins jobs configuration at the level of TestNG test suites.
For that follow this guide: Create a Job section of <https://zebrunner.github.io/community-edition/user-guide/#create-a-job>
2. Register your test project in Jenkins completing steps from Register Organization, Register Repository sections under <https://zebrunner.github.io/community-edition/config-guide/#register-repository>
3. Register your GGR url as default Selenium host for your Jenkins organization (registered in step 2).
Job is located by path /job/Management_Jobs/job/RegisterHubCredentials/
provider = selenium
Pay attention to all properties that you can set on suite level so later they will be applied to your job (provider is one of them): <https://zebrunner.github.io/community-edition/user-guide/#special-annotations>
4. Specify into configuration of your test project `capabilities.provider=selenium`, commit that change, rescan your project with RegisterRepository job
5. Run auto-generated test jobs in Jenkins, make sure that no errors happened. Test should be running on GGR/Selenoid instance. Configuration of generated jobs should come from your TestNG suites.

Materials:

<https://plugins.jenkins.io/job-dsl/>
<https://zebrunner.github.io/community-edition/config-guide/>
<https://zebrunner.github.io/community-edition/user-guide/>

Task 3

Practice with cron (regression pipelines) jobs

Details:

1. Regression pipelines (crons) are needed in order to configure multiple sets of settings for one Jenkins job. Set up cron configuration which will execute your web tests on different browsers (Chrome and Firefox). You'll need to configure configuration matrix for that. Get guide under <https://zebrunner.github.io/community-edition/user-guide/#cron-jobs-layer-of-testing>
2. Commit your config and re-scan your repo in Jenkins (with RegisterRepository). After that you should see a cron job appeared in Jenkins.
3. Run your cron job. Under the hood it should start your Web tests job 2 times but with different browser parameters.

Materials:

Task 4

Practice with webhook setting up

Details:

1. To make your local Jenkins visible for github set up <https://ngrok.com/> on your computer and map local Jenkins url to temporary ngrok public url. Ngrok builds tunnel to your local port and makes it reachable publicly (in our case makes it reachable for github servers)
2. Configure webhook integration for your test repo. Follow this guide: <https://zebrunner.github.io/community-edition/integration/scm/#configure-webhooks>
It will allow Jenkins to monitor state of your test repo and run jobs automatically depending on events from your repo.
3. Make some changes to your project files in github. If everything is configured correctly it will lead to automatic starting of your test job in Jenkins.

Materials:

<https://docs.github.com/en/developers/webhooks-and-events/webhooks/about-webhooks>

Zebrunner

Task 1 (Linux or Windows environment is needed) - optional

Install Zebrunner on Linux or Windows (with WSL2 based Docker Desktop)

Notes:

- Zebrunner is not Mac compatible
- Amazon (AWS) Linux environment can be requested from your mentor

Details:

1. SSH into AWS linux environment
2. Follow the instructions under <https://github.com/zebrunner/reporting> to install Zebrunner.
3. Log in to zebrunner from browser on your local machine
4. Learn how to set up integration of your test project with Zebrunner following <https://zebrunner.com/documentation/reporting/carina>
5. Once integration on test project level is set configure Zebrunner reporting in your local Jenkins. Call job /Management_Jobs/job/RegisterReportingCredentials/ for that.
6. Execute your web and api tests in Jenkins and find results reported in Zebrunner Test Runs page. Make sure that screenshots and test session videos are getting tracked into zebrunner for WEB jobs.
7. Report at least 10 runs and then follow Dashboard documentation to create your personal dashboard: <https://zebrunner.com/documentation/user-guide/dashboards-and-widgets>

Materials:

<https://zebrunner.com/documentation/>

Task 2

Set up integration of tests with Zebrunner reporting

Notes:

-

Details:

1. Request test Zebrunner account from your RM.

After invitation has been sent you'll receive an email with further instructions. Complete instructions to get logged in to Zebrunner. Use your personal project in Zebrunner for all actions.

Step for RM: request new account for a person in **solvinternal** Zebrunner account from Head of the department.

2. Learn Zebrunner documentation at <https://zebrunner.com/documentation/user-guide/workspace>
3. Learn how to integrate carina-based tests with Zebrunner here: <https://zebrunner.com/documentation/reporting/carina>
4. Make sure that you local web runs are tracked properly with artifacts reported (screenshots). If you have issues with tracking of runs started from IDE than read this documentation and follow its instructions: <https://github.com/zebrunner/java-agent-testng#ide> (you'll need to add special argument to your build arguments list)
5. Analyze reported runs, you need to have at least one failing test (if you don't add some fake failure in your test project); check what feature Zebrunner reports have.
6. Register multiple run and try to build your own Zebrunner dashboard: <https://zebrunner.com/documentation/user-guide/dashboards-and-widgets>
7. Configure launcher: <https://zebrunner.com/documentation/guide/launchers/>
8. Run desktop web tests on Zebrunner grid using configured launcher.
9. Optionally: Set up running your mobile tests on Android emulator using configured launcher
10. Optionally: enable saving of web-sessions video to Zebrunner. Some guidance can be found here: <https://zebrunner.com/documentation/reporting/testng#session-artifacts>

You can use selenoid for that since it has in-built feature with video recording

~~Task 5. Set up Zebrunner launcher~~

~~Task 6. Set up automatic integration of test launch with Zebrunner TCM~~

Materials:

Mobile Cloud

Task (Mac environment is needed)

Set up mobile iOS cloud locally

Details:

1. Install mcloud in Docker <https://github.com/zebrunner/mcloud>
2. Install mcloud-ios per guide on github <https://github.com/zebrunner/mcloud-ios>

Use your local iOS simulator as your iOS device.

If not configured yet set up iOS simulator locally with xcode, you can simctl command to control your emulators from terminal.

Specify your simulator info in devices.txt per documentation in github.

Run your mobile iOS Web tests (from Mobile Automation iOS Task) on your simulator using configured mcloud url. Ansible Command Error on mcloud-agent. When running an Ansible command on mcloud-agent, the following error occurs: "ERROR!

[DEPRECATED]: ansible.builtin.include has been removed. Use include_tasks or..."

- In the file "roles/mac-device/tasks/main.yml", replace "include" with "include_tasks"

The IPA file for WebDriverAgent needs to be created and configured according to mcloud-agent README.MD.

After IPA file creation, in "roles/mac-devices/vars/main.yml" add following:

- Fill "WDA FILE" with the path to the created IPA file.
- Fill "WDA BUNDLEID" with the Bundle Identifier used in Xcode for the IPA build.
- Add "wda file" property to device metadata with path to created IPA file.
- Add "wda bundleid" property to device metadata. (e.g., com.facebook.WebDriverAgentRunnerSomeText.xctrunner)

Materials:

<https://medium.com/xcblog/simctl-control-ios-simulators-from-command-line-78b9006a20dc>

Github Actions

Task 1

Set up Github Actions workflow to run **API** tests for each pull request.

Details:

1. Use any existing carina-based project with API tests.
2. Create a YAML workflow file with proper configurations.
Note: it should run your API tests for each new pull request and commit in this PR.
Make sure your tests are running on the branch with changes (not main)
3. Create a PR and make sure tests are running properly.

Materials:

<https://docs.github.com/en/actions>

Task 2

Set up Github Actions workflow to run **web** tests for each pull request.

Details:

1. Use any existing carina-based project with web tests.
2. Create a YAML workflow file with proper configurations for web testing.
3. Create a PR and make sure tests are running properly.

Materials:

<https://docs.github.com/en/actions>

Useful tools

Testrail

Task

Learn testrail basics and usage in automation pipeline

Notes:

To work with Testrail you should strictly use only solvd-lab network being in Minsk office (or some VPN service to hide your Belarus location) - to prevent possible account locking by Testrail.

Testrail internal credentials could be requested from RM.

When logged in you should work only with your personal baseline in testrail (one with your name in the title).

Details:

- Learn basic documentation: <https://support.gurock.com/hc/en-us/articles/7076810203028-Introduction-to-TestRail#Testplansandconfigurations>
- Move created test cases to Testrail
(https://docs.google.com/document/d/1HoJlInmpZa4S-jbnBacINH6aek-N7U3BxPZ_3B0qveQ/edit#heading=h.ja1d4vklr1n7)
- Add new test suite here: <https://zebrunner.testrail.io/index.php?/projects/overview/6> by clicking Add Test Suite button (**put your name into test suite name**)
- Integrate carina tests with Testrail, set up configuration in Zebrunner
<https://zebrunner.com/documentation/integrations/testrail/>
- **API key for integration can be generated here:**
<https://zebrunner.testrail.io/index.php?/mysettings#> (**put your name as part of the key name**)
- Execute mapped suite in Jenkins with tracking results to Zebrunner: check that test cases details are displayed in Zebrunner;
- Check Testrail side: after the run in Jenkins is done new run should appear in Testrail with all actual results for the run: <https://zebrunner.testrail.io/index.php?/runs/overview/4>

Jira

Task

Learn Jira basics and usage in automation pipeline

Notes:

Jira internal credentials could be requested from RM.

When logged in you should work only within specific EPIC. All tickets created by you should have next pattern: `[STUDENT NAME] TICKET TITLE`

Details:

- Create one Jira task for implementation of automated scripts (from the 1st task) under mentioned Jira Epic
- Add integration of Zebrunner with Jira:
<https://zebrunner.com/documentation/integrations/jira/> (for that generate new Personal token for Jira account - use your username in this token's name)
- Make one of your tests failing and create bug for this situation
- Map failed test to created bug in Jira and check that your run appeared PASSED instead of FAILED (because failing test was mapped with a known issue)

Proxyman tool

Task (mac is needed)

Learn operations with proxy tools and use analyzed data in further test automation

Details:

1. Download and install proxyman for Mac: <https://proxyman.io/>
2. Set up https traffic encoding in proxyman for <https://www.saucedemo.com/> website
3. Set up traffic tracking in proxyman and make log in to this website
4. Analyze snipped traffic and find out which cookies are getting set by website for logged in session
5. Automate Test on Carina:
 - a. Automate a test on Carina to open <https://www.saucedemo.com/> home page with the user already logged in.

Note: There might be issues with redirection, requiring manual intervention.

5.1 Set Proper Cookies:

- b. Set the proper cookies for the session to simulate the logged-in state.

5.2 Open Inventory Page:

- c. Open the page <https://www.saucedemo.com/inventory.html> after setting the cookies.

Note: Due to potential redirection issues, manual intervention might be necessary for some steps.

6. Once logged in automate checkout flow in the test.

Browsermob Proxy (analytics test automation)

Task

Lean proxy usage for automation of analytics tests

Details:

1. Read basic information about website analytics usage:
<https://www.monsterinsights.com/reasons-why-website-analytics-is-important-for-your-business-growth/>
2. Learn carina documentation about BrowserMob proxy usage:
<https://zebrunner.github.io/carina/advanced/proxy/>
3. Navigate to <https://www.solvd.com/> in browser and analyze traffic which is getting sent by website (with help of network tab or with proxyman tool). You need to find request to <https://rest.happierleads.com>.
4. Then the goal will be to automate the test which will make sure that website always sends event to this resource. Test should include next steps:
 - Start WebDriver session in your carina project with proxy set to the session.
 - Open solvd website url in your session
 - Automate verification that will call your proxy object in Java and will find mentioned call (to <https://rest.happierleads.com>) in tracked data.
 - Validate found call: payload should be a Json which contains non-empty 'data' property.

The screenshot displays a network monitoring interface. On the left, a list of network requests is shown, including various JavaScript files and tracking events. The right pane provides a detailed view of a selected request, showing its general information and response headers.

Name	Headers	Payload	Preview	Response	Initiator	Timing
8482-ab0333d0/bf0c21a.js						
collect?v=2&tid=G-CZYQYB58BC>m=45je32m0&p=15219.....						
8392-6532af2091aeda20.js						
collect?t=dc&aip=1&r=3&v=1&_v=j99&tid=UA-14268396...1893...						
collect?v=1&_v=j99&a=1521957410&t=pageview&s=1&dl...n81...						
3107-113e1a638f6d657a.js						
new-tracking						
?sid=bElvO73DWzM4ZMqI&data=eyJnYVRyYWNraW5nSWRzljp...						
collect						
129 requests 1.6 MB transferred 4.9 MB resources Finish: 17:						

General

Request URL: <https://rest.happierleads.com/v3/new-tracking>

Request Method: POST

Status Code: 200

Remote Address: 172.67.167.160:443

Referrer Policy: strict-origin-when-cross-origin

Response Headers

access-control-allow-credentials: true

access-control-allow-headers: X-Requested-With, content-type

Advanced Java

Lambda functions

Task 1 (if was not covered in laba)

Basic practice with Lambda functions

Details:

1. Complete free udemy course <https://www.udemy.com/course/java-lambda-expression-video-tutorial/>
2. Add lambda functions usage into your test project for processing of web elements on the pages
3. Create custom Lambda functions with generics and also use it in your page or test classes in test project

Materials:

<https://docs.oracle.com/javase/tutorial/java/javaOO/lambdaexpressions.html>
https://www.w3schools.com/java/java_lambda.asp

Task 2

Improve skills of lambdas usage

Details:

1. Complete all 15 tasks from the document:
<https://docs.google.com/document/d/1jvts07HpXVxzE4toiddj92yP8caa9GneUI8AWbMDBt4/edit>
2. [Optional] Use online [Java compiler](#) instead of IDE for this

Threads

Task (if was not covered in laba)

Practice with threads and pool of threads usage

Details:

1. Create 2 Threads using Runnable and Thread.
2. Create Connection Pool. Use collection from java.util.concurrent package. Connection class may be mocked. The pool should be threadsafe and lazy initialized.

3. Initialize pool with 5 sizes. Load Connection Pool using threads and Thread Pool(7 threads). 5 threads should be able to get the connection. 2 Threads should wait for the next available connection. The program should wait as well.
4. Implement 3rd point but with IFuture and CompletableStage usage.

Materials:

<https://docs.oracle.com/javase/tutorial/essential/concurrency/runthread.html>

https://www.w3schools.com/java/java_threads.asp

Regular expressions **NEW**

Task 1

Details:

1. Write a Java method that validates whether an input string is a valid email address and matches the following pattern: [automation XXXXXXXXX YYYYYY@solv.d.com](#) where X - current date, Y - random letters or figures. For example, [automation 20240325 a1b2c3@solv.d.com](#).

For example:

input:

```
"automation_20240325_a1b2c3@solv.d.com"
```

output:

```
true
```

input:

```
"automation_test_email@solv.d.com"
```

output:

```
false
```

Task 2

Details:

1. Write a Java method that extracts phone numbers from a given text.

For example:

input:

```
"Contact us at +1 (123) 456-7890 or 1-777-123-4567"
```

output:

```
"11234567890", "17771234567"
```

☒

Task 3

Details:

1. Write a Java method that extracts values from a simple JSON and works with different types.

Note: only regular expressions should be used here.

For example:

☒input:

```
String json = "{\"key1\": 11, \"key2\": 'asd', \"key3\": 342}";  
String keyToExtract = "key3";
```

output:

```
342
```

☒

Task 4

Details:

1. Write a Java method that extracts value from a String between square brackets.

For example:

☒input:

```
String input = "ignore me [extract:me] ignore me!";
```

output:

```
"extract:me"
```

☒

Task 5

Details:

1. Write a Java method that groups repeating characters in a String.

For example:

☒input:

```
String input = "qqqqqqweeeeerrtyyyyyyUUUUuuTT";
```

output:

```
"qwertyuit"
```

☒

Materials:

https://www.w3schools.com/java/java_regex.asp

<https://regex101.com/>

Manual testing basics

Task 1

Pass internal course to learn manual testing basics

Details:

1. Pass [internal manual testing course](#).
2. Learn ISTQB materials [ISTQB basic course materials \[EN version\]](#) / [ISTQB basic course materials \[RU version\]](#)

Output:

Next artifacts should be created. Create google doc/drive folder and share it with your mentor:

- Authentication cookies for some website obtained in Chrome
- Bug report
- Checklist
- Test cases (5)
- Simple Test plan
- Postman collection (5 calls) - for API Automation task from this plan
- Tracked network logs from native mobile application (by Charles)

Materials:

- [ISTQB basic course materials \[EN version\]](#)
- [ISTQB basic course materials \[RU version\]](#)

Web Automation Cypress/Webdriver IO

Note: program is only for limited amount of employees. Confirm participation with RM

Task 1

Learn cypress basics via udemy course

Details:

Pass udemy course

<https://www.udemy.com/course/cypress-web-automation-testing-from-zero-to-hero/>

Reach out to your RM for udemy credentials to use

Task 2

Pass internal cypress course to learn about real project oriented cypress usage

Details:

Course program with detailed info can be found by link

<https://docs.google.com/document/d/1AGSyLlHLvG31Nobfgc4GqnlcCC87YcwaW7h9ifA1ohs/edit>

Task 3

Pass internal Webdriver IO course

Details:

Course program with detailed info can be found by link

<https://docs.google.com/document/d/15ABAmXhLMRxcXOxOwpBfFrEnH4ZmvtnGTi5v9n4AY8M/edit>

Performance testing with Gatling

Note: program is only for limited amount of employees. Confirm participation with RM

Follow materials from doc below: [Gatling / Scala](#)

Python/PyTest program

Note: program is only for limited amount of employees. Confirm participation with RM

Follow materials from doc below: https://solvdportal-my.sharepoint.com/:w/r/personal/akaravikou_solvd_com/_layouts/15/Doc2.aspx?action=edit&sourcedoc=%7B7f13aa01-c9cb-4597-9192-11fc914100c4%7D&wdOrigin=TEAMS-MAGLEV.undefined_ns.rwc&wdExp=TEAMS-TREATMENT&wdhostclicktime=1741620705371&web=1